HAC: Simple Hybrids, Smooth Schedules — Rethinking Adaptive Fusion for Resilient UAV Swarm Connectivity Recovery

Abstract

Self-healing connectivity recovery for unmanned aerial vehicle (UAV) swarms requires balancing two competing objectives after partial node failure: restoring network connectivity as quickly as possible, and preserving wide communication coverage. Peng et al. (2026) recently proposed R2C, a resilient recovery framework that divides this problem into an intra-subnetwork virtual-force expansion module, a subnetwork-level multipartite graph convolution module, and a time-adaptive fusion strategy (AWF) that blends the two. In this work, we systematically ablate the design space that R2C occupies along three axes – decision paradigm (heuristic / learned / hybrid), spatial reasoning granularity (node-level / subnetwork-level / global-level), and connectivity–coverage trade-off strategy (fixed / staged / adaptive) – through a sequence of controlled experiments. We show that a markedly simpler hybrid architecture, combining a parameter-free virtual-force branch with a single global graph convolution branch fused at the velocity level, matches or exceeds the robustness of considerably more complex learned baselines, and retains 88–91% of R2C's reported coverage under 80% node failure despite omitting R2C's boundary-node extraction and multipartite graph construction. We further show that naively porting R2C's adaptive fusion schedule to this simplified architecture degrades robustness rather than improving it, trace the failure to the steepness (not the timing) of the fusion weight's transition, and both empirically and theoretically explain why: the transition schedule acts as an implicit training curriculum for the learned branch, and a steep transition forces the network to learn under a harder, less forgiving regime. We validate every finding across two deployment scenarios and five random seeds, and additionally report a falsified hypothesis and a documented numerical bug fix as part of an honest account of the research process. Code and full experimental logs are released to support reproducibility.

1 Introduction

1.1 Problem and Motivation

1.1.1 The UAV Swarm Self-Healing Connectivity Recovery Problem

Unmanned aerial vehicle (UAV) swarms are an increasingly attractive platform for rapid, flexible wireless communication in settings such as post-disaster relief, dynamic battlefields, and temporarily congested areas (Coletta et al., 2023; Shi et al., 2025; Wang et al., 2022, 2024b). A recurring operational risk in these deployments is *partial node failure*: internal faults or external attacks can remove a subset of UAVs from the swarm at any time (Wang et al., 2024a; Kurunathan et al., 2023). When enough nodes fail, the remaining swarm's communication graph can fracture into several disconnected subnetworks, and both the quality of the communication service and the spatial coverage of the mission area degrade sharply (Li et al., 2023; Xu et al., 2022; Peng et al., 2022). The *self-healing communication connectivity* (SCC) problem asks how to reposition the surviving

UAVs, using only local or partially-observed information, so that (i) the swarm's communication graph becomes fully connected again as quickly as possible, and (ii) the swarm continues to provide as much spatial coverage as possible while doing so.

1.1.2 The Inherent Tension Between Connectivity and Coverage

These two objectives are in direct tension. Pulling surviving nodes closer together accelerates reconnection but shrinks the covered area; spreading nodes out preserves coverage but slows – or in the worst case prevents – reconnection. Managing this trade-off, especially as the fraction of failed nodes grows large, is the central technical difficulty of the SCC problem, and is the difficulty that motivates essentially every method surveyed in Section 2.

1.2 Limitations of Existing Approaches

1.2.1 Heuristic Methods Do Not Scale

Non-learning SCC methods rely on hand-designed rules: identifying and compensating for critical nodes (Akkaya et al., 2009; Mi and Yang, 2015; Zhang et al., 2024), or guiding surviving nodes toward predefined rendezvous points (Joshi, 2012; Shriwastav, 2018; Chen et al., 2020; Wang et al., 2024c). These methods are conceptually simple and require no training data, but they depend on static assumptions and hand-crafted coordination rules that make them brittle under large-scale, dynamic, or previously unseen failure patterns (Tosun et al., 2024).

1.2.2 Learned Methods Suffer from Spatial Collapse

Graph neural network (GNN) based methods (Mou et al., 2022; Peng et al., 2025; Wang et al., 2025b) recast SCC as a learned graph optimization problem and have demonstrated faster recovery and stronger adaptability than heuristic baselines. However, these methods are prone to *spatial collapse*: GNN aggregation biases surviving UAVs to concentrate around high-degree nodes, producing an unbalanced spatial distribution and substantial coverage loss. Diffusion-based and differential-subgraph-based methods (Lin et al., 2024; Lin and Ding, 2025) partially alleviate this by encouraging spatially distributed recovery, but still struggle to achieve a stable, balanced layout under dynamic conditions, and generally do not model real-time collision avoidance.

1.3 Our Approach and Findings

1.3.1 A Natural Question Raised by R2C's Architecture

Peng et al. (2026) address this connectivity–coverage trade-off with R2C, which combines a buffered virtual-force field for intra-subnetwork expansion (ISE) with a multipartite graph convolution module for inter-subnetwork translation (IST), fused with a time-adaptive weighting scheme (AWF). R2C reports strong empirical results, but its design bundles together several independent choices – *should* the learned component operate at subnetwork granularity rather than globally, and *does* the adaptive fusion schedule need the specific sigmoid parameters R2C reports, or would any reasonable adaptive schedule do? These questions are not addressed by R2C's own ablations, which toggle whole modules on and off rather than isolating the granularity and weighting-schedule axes independently.

1.3.2 Summary of Findings

We answer these questions with a sequence of controlled experiments (Sections 4–9) built around a deliberately simplified hybrid architecture we call **HAC** (Hybrid Adaptive Convolution): a parameter-free virtual-force branch identical in form to R2C's ISE, fused at the velocity level with a single *global* graph convolution branch – i.e., explicitly *without* R2C's subnetwork partitioning, boundary-node extraction, or multipartite graph construction. Our central findings are:

1. A purely global learned branch, when fused with the heuristic branch under a *fixed* 50/50 weighting, already recovers 88–91% of R2C's reported coverage under 80% node failure, and does so with substantially lower variance than any single-branch baseline we test (Section 8).
2. Naively porting R2C's adaptive fusion schedule (Eq. 22 in Peng et al. (2026)) to this simplified architecture *degrades* robustness, reversing the direction of R2C's own reported ablation (Section 9).
3. The cause is not *when* the fusion weight shifts control to the learned branch, but *how steeply* it does so; smoothing the transition (without changing its timing) restores most of the lost robustness (Section 9), and we give a semi-formal account of why in terms of an implicit training curriculum (Section 6).
4. This fix does *not* transfer to a subnetwork-level learned branch with rigid intra-subnetwork translation, which fails catastrophically under high failure rates regardless of schedule smoothness – cleanly separating a *schedule* failure mode from an *architectural expressiveness* failure mode (Section 9).

1.4 Contributions

1. A three-axis taxonomy of the SCC design space (decision paradigm, spatial granularity, trade-off strategy) that positions R2C and thirteen prior methods, and identifies the specific combination R2C occupies as previously unexplored (Section 3).
2. HAC, a hybrid architecture that is substantially simpler than R2C along the spatial-granularity axis, together with five-seed statistical evidence across two deployment scenarios that it matches or exceeds every single-branch baseline we test, with markedly lower variance (Sections 5, 8).
3. A diagnosis of why R2C's adaptive fusion schedule fails when transplanted into a simplified architecture, together with a validated fix (smoothing the schedule) and a semi-formal implicit-curriculum explanation with an explicit, falsifiable prediction for future work (Sections 6, 9).
4. A fully transparent account of the negative results, a falsified hypothesis, and a numerical implementation bug encountered during this study, together with released code and experimental logs, in the belief that this transparency is itself useful to practitioners building similar hybrid recovery systems (Section 10, Appendix A).

1.5 Paper Organization

Section 2 surveys related work and situates it in our taxonomy. Section 3 formalizes the SCC problem and the taxonomy itself. Section 4 presents two motivating experiments showing the complementary failure modes of single-branch designs. Section 5 describes HAC. Section 6 gives our theoretical analysis of the fusion schedule. Sections 7–9 present the experimental setup, main results, and ablations. Section 10 discusses limitations and future work, and Section 11 concludes.

2 Related Work

2.1 Non-Learning Self-Healing Methods

2.1.1 Critical-Node Compensation

One line of non-learning work identifies critical nodes – those whose failure disconnects the network – and restores connectivity through cascaded movements of their neighbors (Abbasi and Younis, 2008; Akkaya et al., 2009; Mi and Yang, 2015; Zhang et al., 2024). These methods can be effective for small, targeted failures, but the search for critical nodes and the coordination of cascaded repositioning both incur computational costs that scale poorly with swarm size, limiting their use to small-scale failure scenarios.

2.1.2 Rendezvous-Point Aggregation

A second line of work sidesteps explicit critical-node identification by guiding surviving nodes or entire subnetworks toward predefined rendezvous points. Representative instances include Coulomb-force-inspired motion planning (Joshi, 2012), negotiation-based coordination (Shriwastav, 2018), and artificial-potential-field-based aggregation (Chen et al., 2020; Wang et al., 2024c). These methods are conceptually simple and require no training, but rely on static assumptions and hand-crafted coordination rules that generalize poorly to dynamic, uncertain failure patterns (Tosun et al., 2024).

2.2 Learning-Based Self-Healing Methods

2.2.1 GNN-Based Relaxed and Attention Models

The SCC problem can be posed as a graph optimization problem, motivating the use of GNNs to learn recovery strategies directly from data (Peng et al., 2025; Karalias and Loukas, 2020). Mou et al. (2022) model the damaged topology with a relaxed graph convolutional network over virtual communication links; Guo et al. (2024) extend this with spatiotemporal attention. These methods achieve faster recovery and stronger adaptability than heuristic baselines, but are prone to *spatial collapse*: GNN aggregation biases cause UAVs to over-concentrate around high-degree nodes, producing an unbalanced spatial distribution and substantial coverage loss – a failure mode we reproduce directly in Section 4.

2.2.2 Diffusion- and Differential-Subgraph-Based Methods

To mitigate spatial collapse, more recent work incorporates diffusion-based propagation (Lin et al., 2024) or bipartite convolution over multi-hop differential subgraphs (Lin and Ding, 2025), both of which explicitly encourage spatially distributed recovery. These approaches improve spatial balance but still struggle to maintain global stability under dynamic conditions and generally overlook real-time collision avoidance, limiting practical deployment.

2.3 Hybrid Heuristic–Learning Architectures

2.3.1 R2C: Divide-and-Conquer with Adaptive Fusion

Peng et al. (2026) propose R2C, which combines a buffered dynamic virtual force field for intra-subnetwork expansion (ISE) – categorizing pairwise distances into repulsive, neutral, and attractive zones – with a multipartite graph convolution module for inter-subnetwork translation (IST), which extracts convex-hull boundary nodes per fractured subgraph (Barber et al., 1996) and constructs

virtual cross-subgraph edges to predict subnetwork-level translation vectors via a first-order Chebyshev spectral graph convolution (Liu and Wang, 2025). The two branches are combined through an adaptive weighted fusion (AWF) mechanism: a sigmoid-shaped, time-dependent weighting function (Elfwing et al., 2018) that shifts control from the local expansion branch to the global translation branch as recovery progresses. This work is, to our knowledge, the first to explicitly fuse a local virtual-force field with a global graph-convolution-based translation mechanism for SCC, and reports the strongest empirical results among the methods we survey.

2.3.2 Other Hybrid Recovery and Coordination Methods

Li et al. (2025) combine distributed multi-agent reinforcement learning (MADDPG) with an artificial potential field term, but apply the hybridization at node-level granularity throughout, without R2C's subnetwork-level translation branch or its adaptive fusion schedule. Wang et al. (2025b) apply graph-based deep reinforcement learning to topology self-healing without an explicit heuristic component. More broadly, related coordination and coverage problems in multi-robot systems (Mutzari et al., 2025; Ye et al., 2022; Shi et al., 2025) and graph neural network research more generally (Kipf and Welling, 2016; Zeng et al., 2023; Liu and Wang, 2025) inform the architectural choices available at each layer of the design space we formalize next.

2.4 Positioning of This Work

2.4.1 A Three-Axis Taxonomy

The methods surveyed above differ along (at least) three largely independent axes: the *decision paradigm* used to generate motion commands (heuristic, learned, or hybrid), the *spatial reasoning granularity* at which that paradigm operates (node-level, subnetwork-level, or global-level), and the *connectivity–coverage trade-off strategy* used to reconcile the two competing objectives (fixed, staged, or adaptive). We formalize this taxonomy in Section 3.3 and use it to organize both prior work and our own experiments.

2.4.2 Where This Work Sits

R2C occupies the *hybrid* $\times$ {*node-level*, *subnetwork-level*} $\times$ *adaptive* region of this taxonomy – to our knowledge the first work to do so. Its own ablations (*w/o ISE*, *w/o IST*, *w/o AWF*) toggle entire modules but do not independently vary the *granularity* of the learned branch or the *steepness* of the adaptive schedule. This paper fills exactly that gap: Sections 4–9 isolate each axis in turn, starting from single-branch baselines (Section 4), through a deliberately simplified hybrid architecture (Section 5), to a systematic study of the fusion schedule itself (Section 9).

3 Preliminaries and Problem Formulation

3.1 UAV Swarm Network Model

3.1.1 Temporal Graph Representation and Connectivity

Following Peng et al. (2026), at each discrete time step t the UAV swarm is modeled as a temporal graph $G_t = (V_t, E_t, P_t)$, where $V_t = \{v_{1,t}, \ldots, v_{|V_t|,t}\}$ is the set of surviving UAVs, E_t is the set of communication links, and $P_t = \{\mathbf{p}_{1,t}, \ldots, \mathbf{p}_{|V_t|,t}\}$ denotes their planar positions. Let $\mathbf{A}_t$ be the adjacency matrix and $\mathbf{L}_t = \mathbf{D}_t - \mathbf{A}_t$ the graph Laplacian. The number of connected components C_t equals the algebraic multiplicity of the zero eigenvalue of $\mathbf{L}_t$; the swarm is fully connected iff

$C_t = 1$. Partial node failures may fracture G_t into $C_t > 1$ disconnected subnetworks, which are the object of the recovery process we study.

3.1.2 Communication Range and Coverage

A direct link exists between v_i and v_j iff their Euclidean distance $d_{ij,t} = \|\mathbf{p}_{i,t} - \mathbf{p}_{j,t}\|_2$ does not exceed the maximum communication range R_c. Following Peng et al. (2026), the inter-UAV channel is modeled as an air-to-air link (Ma et al., 2019; Wang et al., 2025a), and R_c is derived from a free-space path-loss model (Ni et al., 2024). Each UAV additionally provides circular coverage of radius R_{cov}; the swarm's overall coverage ratio at time t is $C_{\text{cov}}(t) = |\bigcup_i \mathcal{C}_i(t)|/|\mathcal{A}|$, where $\mathcal{A}$ is the mission area and $\mathcal{C}_i(t)$ is UAV v_i's covered disk.

3.2 Problem Definition

3.2.1 Recovery Time and the Optimization Objective

Let t_1 be the time of failure and t_r the earliest subsequent time at which $C_{t_r} = 1$. The recovery time ratio is $T_{\text{rec}} = (t_r - t_1)/T$ for a fixed horizon T. Given initial UAV states and failure dynamics, the SCC problem asks for a velocity control policy $\hat{V}_t = \{\hat{\mathbf{v}}_{1,t}, \ldots, \hat{\mathbf{v}}_{|V_t|,t}\}$ minimizing

$$J_c = T_{\text{rec}} + \big(1 - C_{\text{cov}}(t_r)\big). \tag{1}$$

3.2.2 Constraints

Equation (1) is minimized subject to: (C1) eventual connectivity, $C_{t_r} = 1$; (C2) velocity and position limits, $\|\hat{\mathbf{v}}_{i,t}\| \le \hat{v}_{\max}$ with $\mathbf{p}_{i,t} \in \mathcal{A}$; (C3) explicit position-update dynamics, $\mathbf{p}_{i,t} = \mathbf{p}_{i,t_1} + \sum_{t_1 \le k \le T} \Delta t \cdot \hat{\mathbf{v}}_{i,k}$; and (C4) pairwise communication feasibility, $d_{ij,t} \le R_c$ for any link the policy relies on. Objective (1) makes explicit the tension we study throughout this paper: T_{rec} is minimized by pulling nodes together, while $1 - C_{\text{cov}}(t_r)$ is minimized by spreading them apart.

3.3 A Taxonomy of the Design Space

We organize the space of SCC methods – and our own experiments – along three axes that we argue are mutually exclusive and jointly exhaustive of the design choices a method must make.

3.3.1 Axis 1: Decision Paradigm

A method's motion-command generator is either **heuristic** (hand-designed physical or graph-theoretic rules with no learnable parameters), **learned** (a parametric model trained on data or via differentiable simulation), or **hybrid** (an explicit fusion of a heuristic and a learned component). These three cases are exhaustive: any method's core motion generator is driven by rules alone, by a trained model alone, or by an explicit combination of both.

3.3.2 Axis 2: Spatial Reasoning Granularity

A method reasons about, and produces motion commands at, one of three spatial scales: **node-level** (each UAV acts on local neighbor information), **subnetwork-level** (each connected component of the post-failure graph is treated as the unit of reasoning and produces a single coordinated action), or **global-level** (the entire damaged swarm is reasoned about jointly, without being restricted to local neighborhoods or a subnetwork partition).

Table 1: A subset of the $3 \times 3 \times 3$ design space, showing prior work and the architectures constructed in this paper. ISE/IST refer to R2C's intra-subnetwork expansion and inter-subnetwork translation modules; GAC/SAC/HAC/HSC are the architectures introduced in Sections 4–9.

Paradigm	Granularity	Trade-off	Representative method(s)
Heuristic	Node-level	Fixed	Akkaya et al. (2009); Mi and Yang (2015); Wang et al. (2024c)
Heuristic	Subnetwork	Fixed	Chen et al. (2020)
Heuristic	Global	Fixed	Lin and Ding (2025)
Heuristic	Global	Staged	HERO (Mi et al., 2011)
Learned	Node-level	Fixed	Mou et al. (2022); **GAC** (§4)
Learned	Subnetwork	Fixed	Lin et al. (2024); **SAC** (§4)
Learned	Global	Fixed	Guo et al. (2024)
Hybrid	Node-level	Fixed	Li et al. (2025)
Hybrid	Node+Subnet	Adaptive	R2C (ISE+IST+AWF) (Peng et al., 2026)
Hybrid	**Global**	**Fixed / Adaptive**	**HAC (this work, §5)**
Hybrid	**Subnetwork**	**Fixed / Adaptive**	**HSC (this work, §9)**

3.3.3 Axis 3: Connectivity–Coverage Trade-off Strategy

Finally, a method resolves the tension in Eq. (1) through a **fixed** weighting (a constant trade-off, chosen once, between connectivity- and coverage-oriented behavior), a **staged** strategy (connectivity and coverage objectives are pursued in a strict temporal sequence), or an **adaptive** strategy (the trade-off is continuously re-weighted as a function of the recovery state or elapsed time). R2C's AWF mechanism and our own fusion schedule (Section 5) both fall in this last category.

3.3.4 The Combinatorial Design Space

These three axes define a $3 \times 3 \times 3$ space of candidate SCC designs. Table 1 positions R2C and the prior work surveyed in Section 2 within this space; cells marked *(this work)* are the combinations we construct and evaluate in Sections 4–9.

4 Motivating Experiments: Failure Modes of Single-Branch Designs

Before introducing our hybrid architecture, we establish empirically that purely global and purely subnetwork-level *learned* branches each fail in a distinct, complementary way. These two architectures also serve as baselines throughout the rest of the paper.

4.1 Global-Level Learned Policy (GAC)

4.1.1 Architecture and Training

GAC (Global Adaptive-weighted Convolution) is a single graph convolutional policy operating over a k-nearest-neighbor graph built on *all* surviving UAVs in Euclidean space (not restricted to nodes within communication range R_c, so that information can propagate across disconnected subnetworks). The policy predicts a per-node 2D velocity directly; there is no subnetwork partition and no separate node/subnetwork branches to fuse. GAC is trained end-to-end via a differentiable

multi-step rollout with a connectivity surrogate loss, a coverage surrogate loss, and a movement-cost loss (the same loss family used for HAC's learned branch; see Section 5.5).

4.1.2 Results: Spatial Collapse and Low Success Rate

Table 2 (columns 2–3) reports GAC's connectivity recovery success rate (fraction of 50 trials, across 5 random seeds, in which full connectivity is restored within the recovery horizon T) and mean node degree (AND, a proxy for spatial clustering) under the random deployment scenario. GAC's success rate degrades sharply as the failure rate increases, reaching only 19.8 ± 5.5 out of 50 trials at 80% failure, with an elevated average node degree (13.7 at 20% failure) that is symptomatic of the spatial-collapse failure mode reported for GNN-based methods in the literature (Section 2).

4.2 Subnetwork-Level Rigid Translation (SAC)

4.2.1 Architecture and Training

SAC (Subnetwork-level Adaptive Convolution) partitions surviving UAVs into subnetworks by connected component under the communication-range graph, computes one pooled embedding per subnetwork (centroid position and relative size), and applies a graph convolution over a k-nearest-neighbor graph of subnetwork *centroids* to predict a single 2D translation vector per subnetwork. By explicit design choice, all UAVs within a given subnetwork share the exact same translation velocity for that step (pure rigid translation, with no per-node local adjustment layered on top). This isolates the spatial-granularity axis as cleanly as possible: SAC is identical to GAC in every respect (loss functions, training procedure, graph convolution backbone hyperparameters) except that its motion commands are generated, and its message passing occurs, at subnetwork rather than node granularity.

4.2.2 Results: Reduced Clustering, Collapsed Recovery Capability

Table 2 (columns 4–5) shows the opposite failure mode: SAC achieves a substantially lower, more uniform average node degree than GAC at every failure rate (e.g., 6.8 vs. 13.7 at 20% failure), confirming that subnetwork-level rigid translation does mitigate spatial collapse. However, SAC's connectivity recovery success rate collapses even faster than GAC's as the failure rate increases, reaching 5.0 ± 0.0 out of 50 at 60% failure and 0.0 ± 0.0 at 80% failure – a deterministic failure (zero variance across all 5 seeds), not a rare or stochastic event. Rigid translation cannot compensate for large numbers of small, widely separated subnetworks, which the pure global average node degree metric cannot express: a subnetwork of one node still translates as a rigid body.

4.3 Takeaway: Complementary Failure Modes Motivate Hybridization

GAC and SAC fail in opposite, complementary ways: GAC recovers connectivity more often but at the cost of severe spatial clustering; SAC preserves a uniform spatial distribution but cannot reliably reconnect fragmented subnetworks under moderate-to-severe failure. Neither failure mode is fixed by more training or a different random seed – both are architectural properties of operating at a single, fixed spatial granularity with a purely learned decision paradigm. This observation directly motivates hybridizing a learned branch with a heuristic branch (Section 5), and later motivates asking *which* granularity the learned branch in such a hybrid should operate at (Section 9).

Table 2: Motivating experiments (random deployment, mean ± std over 5 seeds): GAC (global-level learned) exhibits spatial collapse (elevated AND) and degrading success rate; SAC (subnetwork-level rigid translation) reduces clustering but its recovery capability collapses deterministically under high failure rates. Success is the number of trials (out of 50) that reach full connectivity within the recovery horizon.

Failure	GAC (global)		SAC (subnetwork)	
rate	AND	Success/50	AND	Success/50
20%	13.68 ± 0.28	49.0 ± 0.6	6.82 ± 0.00	40.0 ± 0.0
40%	12.48 ± 0.61	47.2 ± 1.7	5.85 ± 0.00	26.0 ± 0.0
60%	10.55 ± 0.78	42.8 ± 3.7	4.18 ± 0.00	5.0 ± 0.0
80%	6.00 ± 0.26	19.8 ± 5.5	n/a	0.0 ± 0.0

5 Method: Hybrid Adaptive Convolution (HAC)

5.1 Overview and Design Principles

HAC is deliberately designed to be the simplest possible instantiation of the *hybrid* × *global* cell in Table 1: a parameter-free heuristic branch, identical in form to R2C's ISE module, fused at the *velocity* level with a single global graph convolution branch – explicitly without R2C's subnetwork partitioning, convex-hull boundary-node extraction, or multipartite graph construction. This minimalism is intentional: it lets us attribute any performance gap between HAC and R2C (Section 8.2) specifically to the architectural machinery HAC omits, rather than to confounded differences in loss design, training procedure, or hyperparameters.

5.2 Heuristic Branch: Buffered Virtual-Force Field

5.2.1 Three-Zone Force Model

We reproduce R2C's buffered virtual-spring force field (Rychlewski, 1984) without modification. For an ideal inter-UAV distance range $[d_l, d_u] \subset (0, R_c]$, the force on UAV v_i from a neighbor v_j within communication range is

$$\mathbf{F}'_{ij,t} = \begin{cases} -\exp\left(-\alpha \cdot \dfrac{d_{ij,t}}{d_l}\right) \mathbf{u}_{ij,t}, & 0 \le d_{ij,t} < d_l \\ \mathbf{0}, & d_l \le d_{ij,t} \le d_u \\ \exp\left(-\beta \cdot \dfrac{R_c - d_{ij,t}}{R_c - d_u}\right) \mathbf{u}_{ij,t}, & d_u < d_{ij,t} \le R_c, \end{cases} \tag{2}$$

where $\mathbf{u}_{ij,t} = (\mathbf{p}_{j,t} - \mathbf{p}_{i,t})/d_{ij,t}$ is the unit vector from v_i toward v_j, and $\alpha, \beta > 0$ control the decay rates of the repulsive and attractive zones respectively. We use $\alpha = \beta = 0.01$ and $[d_l, d_u] = [0.8R_c, 0.9R_c]$, matching R2C's reported parameter settings.

5.2.2 Boundary Repulsion

To keep UAVs within the deployment area $\mathcal{A}$, we add an inward boundary-repulsion term for any UAV within a buffer distance d_e of a mission-area edge $k \in \mathcal{B} = \{\text{left,right,bottom,top}\}$: $\mathbf{F}''_{i,t} = \sum_{k \in \mathcal{B}} \exp(-\delta \cdot d_{i,k})\, \mathbf{n}_k$, where $d_{i,k}$ is the distance to boundary k and $\mathbf{n}_k$ its inward normal. The total heuristic force is $\mathbf{F}_{i,t} = \sum_{j \in \mathcal{N}_i} \mathbf{F}'_{ij,t} + \mathbf{F}''_{i,t}$ over neighbors $\mathcal{N}_i$ within communication range, and the

resulting velocity is $\mathbf{v}'_{i,t} = \hat{v}_{\max} \cdot \mathbf{F}_{i,t}/(\|\mathbf{F}_{i,t}\| + \epsilon)$. Because Eq. (2) contains no learnable parameters, this branch requires no training and is identical at training and evaluation time – a property we exploit directly in the theoretical analysis of Section 6.

5.3 Learned Branch: Global Graph Convolution

5.3.1 k-Nearest-Neighbor Graph Construction

At each step, we build a symmetric k-nearest-neighbor graph over the Euclidean positions of *all* surviving UAVs, irrespective of communication range, so that the graph convolution can propagate information across otherwise-disconnected subnetworks. We use $k = 8$ throughout.

5.3.2 Node Features and Architecture

Each node's input feature is a 3-dimensional vector: its normalized (x, y) position and a normalized local degree (computed from the k-NN graph). These are passed through an L-layer graph convolutional network (we use $L = 4$, hidden dimension 128) with ReLU activations, followed by a two-layer MLP head that outputs a 2D direction, passed through tanh and scaled by $\hat{v}_{\max}$ to produce $\mathbf{v}''_{i,t}$.

5.4 Velocity-Level Adaptive Fusion

5.4.1 Sigmoid Schedule

Following R2C's Eq. 22, we define a time-dependent fusion weight

$$w_T(t) = \frac{1}{1 + \exp(-k \cdot (t/T - s_0))}, \qquad w_E(t) = 1 - w_T(t), \tag{3}$$

where T is the recovery horizon, $k > 0$ controls the steepness of the transition, and $s_0 \in (0, 1)$ its center. R2C reports $k = 30$, $s_0 = 0.125$; we treat both as tunable and study their effect systematically in Section 9.

5.4.2 Fusion Formula

The two branches' velocities are combined and renormalized to magnitude $\hat{v}_{\max}$, mirroring R2C's Eq. 23:

$$\hat{\mathbf{v}}_{i,t} = \hat{v}_{\max} \cdot \frac{w_E(t)\,\mathbf{v}'_{i,t} + w_T(t)\,\mathbf{v}''_{i,t}}{\left\| w_E(t)\,\mathbf{v}'_{i,t} + w_T(t)\,\mathbf{v}''_{i,t} \right\| + \epsilon}. \tag{4}$$

Two fixed-weight variants are used throughout as controls: **HAC-Fixed** sets $w_E = w_T = 0.5$ for all t (R2C's own *w/o AWF* ablation design), while **HAC-Adaptive** uses Eq. (3).

5.5 Training Objective

5.5.1 Connectivity Surrogate Loss

The number of connected components used in R2C's own connectivity loss (their Eq. 19) is not differentiable. We instead use a soft-minimum surrogate: for every pair of currently-disconnected

Algorithm 1 HAC Training (one epoch)

Require: Learned policy π_θ, rollout length T, weight schedule $w_T(\cdot)$

Sample failure ratio $r \sim \{0.2, 0.4, 0.6, 0.8\}$; initialize positions $\mathbf{P}_0$; apply failure
$\mathcal{L}_{\text{total}} \leftarrow 0$
for $t = 0, \ldots, T-1$ **do**
    Compute connected components $\{S_k\}$ of $\mathbf{P}_t$ under R_c
    $\mathbf{v}' \leftarrow$ heuristic velocities via Eq. (2) ▷ no gradient
    Build k-NN graph over $\mathbf{P}_t$; $\mathbf{v}'' \leftarrow \pi_\theta(\mathbf{P}_t)$ ▷ learned branch
    $w_T \leftarrow w_T(t)$; $w_E \leftarrow 1 - w_T$
    $\hat{\mathbf{v}} \leftarrow$ fuse $(\mathbf{v}', \mathbf{v}'', w_E, w_T)$ via Eq. (4)
    $\mathbf{P}_{t+1} \leftarrow \text{clip}(\mathbf{P}_t + \Delta t \cdot \hat{\mathbf{v}},\ 0,\ L)$
    $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{total}} + 0.5\,\mathcal{L}_{\text{conn}}(\mathbf{P}_{t+1}) + 0.5\,\mathcal{L}_{\text{cov}}(\mathbf{P}_{t+1}) + \lambda\,\mathcal{L}_{\text{move}}$
end for
Backpropagate $\mathcal{L}_{\text{total}}/T$ through π_θ; update θ via Adam

components S_i, S_j,

$$\mathcal{L}_{\text{conn}}^{(i,j)} = -\tau \left(\log \sum_{\substack{u \in S_i \\ v \in S_j}} \exp\left(-\frac{d_{uv}}{\tau}\right) - \log(n_i n_j) \right), \tag{5}$$

averaged over all component pairs, where $n_i = |S_i|$, $n_j = |S_j|$. The $\log(n_i n_j)$ correction term is necessary: without it, a plain log-sum-exp soft-minimum is not count-invariant, and systematically under-penalizes distance between large components purely because more pairs are summed over (Appendix A documents this bug and its numerical verification).

5.5.2 Coverage and Movement Losses

Coverage is approximated by sampling grid points $\mathbf{g} \in G$ from the mission area and using a smooth indicator of coverage, $\mathcal{L}_{\text{cov}} = \frac{1}{|G|} \sum_{\mathbf{g} \in G} \left(1 - \sigma\left(\frac{R_{\text{cov}} - \min_i \|\mathbf{g} - \mathbf{p}_i\|}{\tau'}\right)\right)$, and movement cost by $\mathcal{L}_{\text{move}} = \frac{1}{N} \sum_i \|\hat{\mathbf{p}}_i - \mathbf{p}_i\|$. For HAC, both connectivity and coverage losses are weighted equally (0.5 each) throughout training; the connectivity–coverage trade-off is realized entirely at the *velocity* level via Eq. (4), not by additionally re-weighting the loss – the key architectural difference from a single-branch policy with adaptively weighted loss terms (which we test as a separate ablation in Section 4-adjacent experiments, see Appendix C).

5.6 Algorithm Summary

Algorithm 1 summarizes HAC's training procedure (a differentiable multi-step rollout with back-propagation through the fused velocity field), and Algorithm 2 summarizes deployment (inference-time rollout, no gradient computation).

6 Theoretical Analysis

Section 9 will show empirically that naively porting R2C's fusion schedule (Eq. (3) with $k = 30$, $s_0 = 0.125$) to HAC *degrades* robustness relative to a fixed 50/50 fusion, and that smoothing the schedule (reducing k) largely restores it. This section gives two complementary explanations for

Algorithm 2 HAC Deployment (inference rollout)

Require: Trained policy π_θ, recovery horizon T, weight schedule $w_T(\cdot)$

for $t = 0, \dots, T-1$ **do**
  if swarm is fully connected **then return** success, t
  end if
  $\mathbf{v}' \leftarrow$ heuristic velocities via Eq. (2)
  $\mathbf{v}'' \leftarrow \pi_\theta(\mathbf{P}_t)$ ▷ no gradient
  $\hat{\mathbf{v}} \leftarrow$ fuse $(\mathbf{v}', \mathbf{v}'', 1 - w_T(t), w_T(t))$ via Eq. (4)
  $\mathbf{P}_{t+1} \leftarrow \mathrm{clip}(\mathbf{P}_t + \Delta t \cdot \hat{\mathbf{v}},\ 0,\ L)$
end for
return failure (horizon exhausted)

why the steepness of the transition, rather than its timing, is the operative variable: a test-time argument based on a monotonicity property of the heuristic branch (Section 6.1), and a training-time argument based on the schedule's role as an implicit curriculum for the learned branch (Section 6.2), which we identify as the more fundamental of the two.

6.1 Local Monotonicity of the Heuristic Branch

Proposition 1 (Attractive-zone descent guarantee). *Let v_i, v_j be two UAVs with $d_u < d_{ij,t} \le R_c$ (i.e., in the attractive zone of Eq. (2)), and suppose no other neighbor's repulsive or boundary force acts on v_i or v_j. Then, for sufficiently small Δt, applying the heuristic branch alone (Eq. (2) with $w_E = 1, w_T = 0$ in Eq. (4)) strictly decreases $d_{ij,t}$.*

Proof. By Eq. (2), the pairwise force on v_i from v_j in the attractive zone is $\mathbf{F}'_{ij,t} = \exp\!\left(-\beta \frac{R_c - d_{ij,t}}{R_c - d_u}\right) \mathbf{u}_{ij,t}$ with $\mathbf{u}_{ij,t}$ pointing from v_i toward v_j; by symmetry, $\mathbf{F}'_{ji,t} = -\mathbf{F}'_{ij,t} \cdot (d_{ij,t}/d_{ji,t}) = -\mathbf{u}_{ij,t} \exp(\cdot)$ points from v_j toward v_i with equal magnitude (since $d_{ij,t} = d_{ji,t}$ and $\mathbf{u}_{ji,t} = -\mathbf{u}_{ij,t}$). Under the stated isolation assumption, $\mathbf{F}_{i,t} = \mathbf{F}'_{ij,t}$ and $\mathbf{F}_{j,t} = \mathbf{F}'_{ji,t} = -\mathbf{F}'_{ij,t}$, so $\mathbf{v}'_{i,t} = \hat{v}_{\max}\, \mathbf{u}_{ij,t}$ and $\mathbf{v}'_{j,t} = -\hat{v}_{\max}\, \mathbf{u}_{ij,t}$ (the normalization in $\mathbf{v}'_{i,t} = \hat{v}_{\max} \mathbf{F}_{i,t} / (\|\mathbf{F}_{i,t}\| + \epsilon)$ preserves direction). The one-step position update gives $d_{ij,t+\Delta t} = \|\mathbf{p}_{i,t} + \Delta t\, \mathbf{v}'_{i,t} - \mathbf{p}_{j,t} - \Delta t\, \mathbf{v}'_{j,t}\| = \|(\mathbf{p}_{i,t} - \mathbf{p}_{j,t}) - 2\Delta t\, \hat{v}_{\max}\, \mathbf{u}_{ij,t}\|$. Since $\mathbf{p}_{i,t} - \mathbf{p}_{j,t} = -d_{ij,t}\, \mathbf{u}_{ij,t}$, this equals $|-d_{ij,t} - 2\Delta t\, \hat{v}_{\max}| = d_{ij,t} - 2\Delta t \hat{v}_{\max}$ for Δt small enough that this remains non-negative, which is strictly less than $d_{ij,t}$ for any $\Delta t > 0$. ■ □

Remark 1. *Proposition 1 gives the heuristic branch a* local, certifiable descent guarantee *on connectivity, at least within the attractive zone and modulo interference from other forces. The learned branch carries no such guarantee – indeed Section 4 shows GAC can* increase *average node degree relative to a more spatially balanced configuration, i.e. move in a direction the connectivity objective does not favor. Because Eq. (4) is a convex combination (before renormalization), there is intuitively a threshold $w_E^{\min}$ (state- and training-progress-dependent) below which the learned branch's contribution can overwhelm the heuristic branch's descent direction. This motivates asking, for a given trajectory, how large a fraction of the recovery process operates with $w_E(t)$ below such a threshold – which the schedule parameters k, s_0 control directly via Eq. (3).*

6.2 Fusion Weight as an Implicit Training Curriculum

The monotonicity argument above is a *test-time* account: it explains why control should not be handed too fully to the learned branch mid-trajectory. But it does not by itself explain why smoothing the schedule (as opposed to, say, delaying it) is what fixes the problem empirically

(Section 9). We argue the more fundamental mechanism is a *training-time* one, exploiting the fact that $\mathbf{v}''$ in HAC is trained *end-to-end* through Eq. (4) (Algorithm 1), rather than pretrained in isolation and fused only at deployment.

Definition 1 (Autonomy exposure)**.** *For a training rollout of length T_{train} and a threshold $\tau \in (0,1)$, the* autonomy exposure *of a schedule $w_T(\cdot; k, s_0)$ is the fraction of training steps at which the learned branch is in majority control:* $\text{AE}(k, s_0; \tau) = \frac{1}{T_{\text{train}}} \big| \{\, t \in \{0, \ldots, T_{\text{train}} - 1\} : w_T(t; k, s_0) > \tau \,\} \big|$.

For $\tau = 0.5$, the sigmoid in Eq. (3) crosses from 10% to 90% over an interval of width $\Delta t \approx \frac{\ln(81)}{k} \cdot T_{\text{train}} \approx \frac{4.394}{k} T_{\text{train}}$ (derived from inverting the sigmoid at the 10%/90% points), so smaller k directly widens the transition window and increases the training-time exposure the learned branch has to *low-w_T* regimes, where the heuristic branch is doing most of the work and the learned branch is effectively performing residual correction rather than autonomous control.

Remark 2 (Curriculum interpretation)**.** *A steep schedule (large k, e.g. R2C's default $k = 30$) confronts the learned branch, for the majority of a training rollout, with a* harder *learning problem: near-autonomous control with no heuristic correction, compounding open-loop error over many steps. A smooth schedule (small k) instead keeps the learned branch in a residual-correction regime for most of training – a form of curriculum learning by easy-to-hard ordering, except here the "easy" and "hard" regimes are not separate training phases but different* fractions of exposure within every single rollout.

6.2.1 Consistency with Observed Data

This account is consistent with the two strongest empirical facts from Section 9: (i) among the schedules we test, the smoothest ($k = 5$, transition width $\approx 88\%$ of the 40-step training rollout) performs best, nearly matching HAC-Fixed at 80% failure; and (ii) this fix does *not* transfer to HSC (the subnetwork-level hybrid variant), whose learned branch is architecturally restricted to per-subnetwork rigid translation – no amount of curriculum smoothing can grant a policy representational capacity it does not have, precisely the boundary condition the curriculum account predicts.

6.2.2 An Honest Gap in the Account

We do not fully explain the observed non-monotonicity between two intermediate schedules (§9: $k = 10, s_0 = 0.125$ outperforms $k = 30, s_0 = 0.5$, in the direction the curriculum account predicts, but is not itself clearly better than R2C's default $k = 30, s_0 = 0.125$ despite a 3× wider transition window). These intermediate points were swept with a single training seed each; given the five-seed variance we later measure for comparable architectures (success-count standard deviations up to ±5–6 out of 50), the ordering among intermediate schedules is plausibly within noise, and we do not treat it as evidence against the curriculum account – nor as evidence for it.

6.3 A Falsifiable Prediction for Future Work

If the autonomy-exposure account is correct, $\text{AE}(k, s_0; 0.5)$ should predict final success-rate ranking *better* than k or s_0 individually – i.e., two schedules with different (k, s_0) but matched AE should perform comparably. This is a concrete, computable, and falsifiable prediction: re-running the schedule sweep of Section 9 with matched autonomy exposure across several seeds, rather than the single-seed sweep reported there, would directly test it.

7 Experimental Setup

7.1 Simulation Environment

7.1.1 Deployment Scenarios

We evaluate in a $1000 \times 1000\,\mathrm{m}^2$ area under two deployment scenarios, mirroring Peng et al. (2026): **random**, with 200 UAVs placed uniformly at random, and **uniform**, with 196 UAVs arranged on a 14×14 grid. Each UAV has communication and coverage radius $R_c = R_{\mathrm{cov}} = 120\,\mathrm{m}$, maximum speed $\hat{v}_{\max} = 1\,\mathrm{m/s}$, and a recovery horizon of $T = 400$ simulation steps at evaluation time.

7.1.2 Failure Model and Independent Trials

We test four failure ratios, $\{20\%, 40\%, 60\%, 80\%\}$, with failed nodes selected uniformly at random (the paper's own failure-selection procedure is not specified beyond "unpredictable random failure patterns"; see Appendix A for this and other documented deviations). For each (architecture, deployment, failure ratio) combination we run 5 independent training seeds $\times$ 50 independent evaluation trials per seed, and report the mean $\pm$ standard deviation across seeds of each trial-level metric's mean.

7.2 Baselines

7.2.1 Heuristic-Only Baseline

The virtual-force branch (Section 5.2) run in isolation (w_E=1, w_T=0 throughout); requires no training.

7.2.2 Learned Baselines

GAC and SAC (Section 4); and a *learned-only* ablation of HAC itself, in which the fusion weight is forced to w_T=1 throughout both training and evaluation (the learned branch receives zero contribution from the heuristic branch at any point).

7.2.3 R2C (Non-Reproduced Reference)

We additionally report R2C's own numbers as published in its Table 1 and Table 2 (Peng et al., 2026). R2C's code is not publicly released; this comparison is explicitly *not* a controlled reproduction and is discussed with that caveat in Section 8.2.

7.3 Evaluation Metrics

We report: **Success/50**, the number of evaluation trials (out of 50) reaching $C_t = 1$ within the horizon T (trials that do not are excluded from the remaining per-trial metrics, following R2C's own convention of marking non-recovering configurations invalid); **REC**, the recovery time ratio T_{rec} averaged over successful trials; **COV**, the coverage ratio $C_{\mathrm{cov}}(t_r)$ at the moment of recovery, averaged over successful trials; and **AND**, the average node degree at the moment of recovery, used as a proxy for spatial clustering (higher AND indicates more clustering, all else equal).

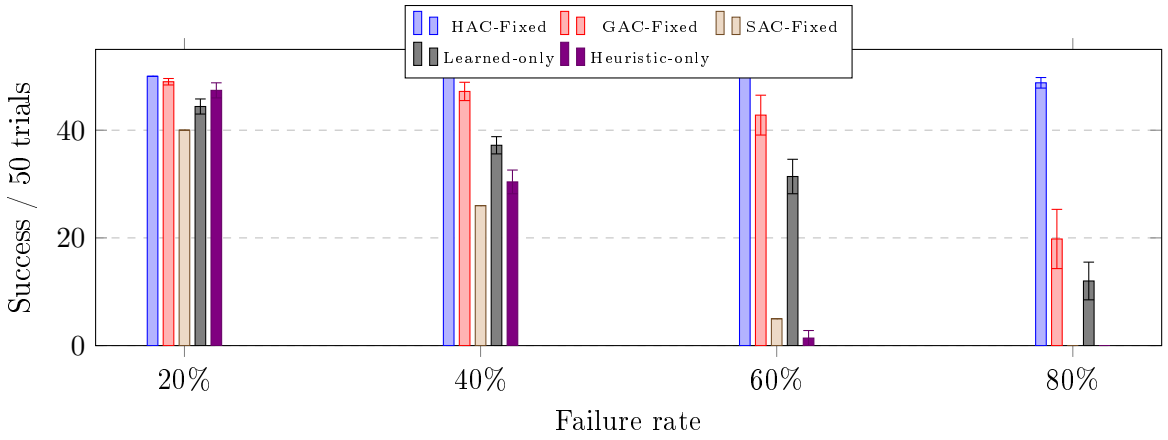


Figure 1: Connectivity recovery success rate (out of 50 trials) vs. failure rate, random deployment, mean ± std over 5 seeds. HAC-Fixed maintains near-perfect success across all failure rates, with markedly lower variance than every other architecture, especially at 80% failure.

7.4 Implementation Details

7.4.1 Network and Optimization Hyperparameters

All graph convolution backbones (GAC, SAC, and HAC's learned branch) use L=4 layers, hidden dimension 128, ReLU activations, and a two-layer MLP head. Training uses Adam with learning rate 10^{-3}, gradient-norm clipping at 1.0, and 300 training epochs, each consisting of a differentiable rollout of 40 steps (substantially shorter than the 400-step evaluation horizon, a documented tractability trade-off; see Appendix A).

7.4.2 Multi-Seed Protocol

An early implementation issue left network weight initialization uncontrolled by the `-seed` argument (only environment-side randomness, e.g. failure selection, was seeded). We identified and fixed this prior to running the five-seed studies reported in Section 8; Appendix A documents the fix and its measurable effect on baseline variance.

8 Results

8.1 Main Results: HAC vs. Single-Branch Baselines

8.1.1 Random Deployment Results

Table 3 reports Success/50, REC, and COV for all five architectures under random deployment. HAC-Fixed dominates every other architecture at every failure rate, and the gap widens sharply as the failure rate increases: at 80% failure, HAC-Fixed succeeds in 48.8 ± 0.98 trials, versus 19.8 ± 5.5 for GAC-Fixed (the next best), 12.0 ± 3.5 for the learned-only ablation, and 0.0 ± 0.0 for both SAC-Fixed and the heuristic-only baseline. Figure 1 visualizes this gap directly from the same data.

Table 3: Main results, random deployment, mean ± std over 5 seeds. Success is out of 50 trials. n/a indicates zero successful trials (REC/COV undefined).

Architecture	Failure	Success/50	REC	COV
HAC-Fixed	20%	50.0 ± 0.0	0.061 ± 0.006	0.997 ± 0.000
	40%	50.0 ± 0.0	0.136 ± 0.006	0.988 ± 0.001
	60%	50.0 ± 0.0	0.307 ± 0.006	0.923 ± 0.005
	80%	$\mathbf{48.8 \pm 0.98}$	0.666 ± 0.032	$\mathbf{0.604 \pm 0.005}$
GAC-Fixed	20%	49.0 ± 0.6	0.324 ± 0.007	0.747 ± 0.009
	40%	47.2 ± 1.7	0.467 ± 0.020	0.614 ± 0.021
	60%	42.8 ± 3.7	0.612 ± 0.027	0.452 ± 0.022
	80%	19.8 ± 5.5	0.775 ± 0.017	0.310 ± 0.009
SAC-Fixed	20%	40.0 ± 0.0	0.216 ± 0.000	0.910 ± 0.000
	40%	26.0 ± 0.0	0.567 ± 0.000	0.713 ± 0.000
	60%	5.0 ± 0.0	0.852 ± 0.000	0.531 ± 0.000
	80%	0.0 ± 0.0	n/a	n/a
Learned-only	20%	44.4 ± 1.4	0.277 ± 0.013	0.851 ± 0.008
	40%	37.2 ± 1.6	0.402 ± 0.009	0.759 ± 0.008
	60%	31.4 ± 3.2	0.613 ± 0.021	0.558 ± 0.019
	80%	12.0 ± 3.5	0.874 ± 0.025	0.306 ± 0.016
Heuristic-only	20%	47.4 ± 1.4	0.051 ± 0.017	0.997 ± 0.000
	40%	30.4 ± 2.2	0.120 ± 0.027	0.993 ± 0.001
	60%	1.4 ± 1.4	0.224 ± 0.091	0.964 ± 0.008
	80%	0.0 ± 0.0	n/a	n/a

8.1.2 Uniform Deployment Results

Table 4 repeats the comparison for uniform deployment. The same ordering holds: HAC-Fixed reaches 49.0±2.0 out of 50 at 80% failure, versus 25.8±6.1 for GAC-Fixed and 0.0±0.0 for both SAC-Fixed and heuristic-only. The SAC-Fixed and heuristic-only failures at 60–80% are deterministic across both deployment scenarios (zero or near-zero variance), indicating an architectural rather than stochastic cause, consistent with Section 4.

8.2 Comparison with R2C

8.2.1 Quantitative Comparison

Table 5 juxtaposes R2C's own reported numbers (Peng et al., 2026) against HAC-Fixed. We stress that this is *not* a controlled reproduction (Section 7.2.3); R2C's training details, random seeds, and exact implementation are not published or released, and our simulation environment is an independent re-implementation of R2C's Definitions 1–5.

8.2.2 Interpretation and Caveats

R2C outperforms HAC-Fixed on most metrics, but the gap is small and not always in R2C's favor (HAC is faster at 20% uniform-deployment failure). Low-failure-rate comparisons are dominated by small-sample effects on absolute recovery time and should not be over-interpreted. The most meaningful comparison is at 80% failure, where the gap is consistent in direction across both deployments: R2C's coverage exceeds HAC-Fixed's by roughly 11–12 percentage points. Given that R2C's learned branch includes convex-hull boundary-node extraction, virtual cross-subgraph edge construction, and an 8-layer, 512-hidden-dimension spectral graph convolution – machinery entirely absent from HAC's single global 4-layer, 128-hidden-dimension convolution – we read this result positively: **HAC-Fixed retains** 88.7% **(random) and** 90.7% **(uniform) of R2C's reported**

Table 4: Main results, uniform deployment, mean ± std over 5 seeds.

Architecture	Failure	Success/50	REC	COV
HAC-Fixed	20%	50.0 ± 0.0	0.003 ± 0.000	1.000 ± 0.000
	40%	50.0 ± 0.0	0.037 ± 0.002	0.998 ± 0.000
	60%	50.0 ± 0.0	0.258 ± 0.012	0.957 ± 0.003
	80%	$\mathbf{49.0 \pm 2.0}$	0.641 ± 0.052	$\mathbf{0.606 \pm 0.005}$
GAC-Fixed	20%	50.0 ± 0.0	0.011 ± 0.003	0.992 ± 0.003
	40%	50.0 ± 0.0	0.200 ± 0.014	0.843 ± 0.019
	60%	47.0 ± 2.8	0.574 ± 0.013	0.478 ± 0.023
	80%	25.8 ± 6.1	0.772 ± 0.057	0.300 ± 0.012
SAC-Fixed	20%	49.8 ± 0.4	0.016 ± 0.006	0.996 ± 0.002
	40%	35.2 ± 3.7	0.256 ± 0.060	0.944 ± 0.017
	60–80%	0.0 ± 0.0	n/a	n/a
Learned-only	20%	50.0 ± 0.0	0.008 ± 0.003	0.998 ± 0.002
	40%	49.2 ± 0.7	0.228 ± 0.022	0.880 ± 0.021
	60%	33.8 ± 6.4	0.625 ± 0.012	0.581 ± 0.032
	80%	11.6 ± 6.5	0.866 ± 0.070	0.337 ± 0.024
Heuristic-only	20%	49.8 ± 0.4	0.006 ± 0.002	1.000 ± 0.000
	40%	44.0 ± 1.3	0.077 ± 0.017	0.998 ± 0.000
	60%	0.8 ± 0.7	0.110 ± 0.081	0.977 ± 0.009
	80%	0.0 ± 0.0	n/a	n/a

Table 5: HAC-Fixed vs. R2C's own reported numbers (Peng et al., 2026). Not a controlled comparison; see Section 8.2 for caveats.

	Random		Uniform	
Failure	R2C REC/COV	HAC REC/COV	R2C REC/COV	HAC REC/COV
20%	0.023 / 0.996	0.061 / 0.997	0.037 / 1.000	0.003 / 1.000
40%	0.070 / 0.989	0.136 / 0.988	0.051 / 0.999	0.037 / 0.998
60%	0.204 / 0.951	0.307 / 0.923	0.181 / 0.978	0.258 / 0.957
80%	0.625 / **0.681**	0.666 / **0.604**	0.617 / **0.668**	0.641 / **0.606**

coverage under the most severe failure condition tested, using a substantially simpler learned component and no learned fusion schedule. This supports our central claim that much of R2C's benefit is attributable to the hybrid heuristic–learned *paradigm* itself rather than to the specific subnetwork-level machinery R2C uses to implement it. The remaining gap is a natural target for future work that reintroduces boundary-aware structure into the learned branch without reverting to full subnetwork partitioning.

8.3 Topology Quality Analysis

8.3.1 Average Node Degree as a Clustering Proxy

Figure 2 plots AND against failure rate for all five random-deployment architectures. HAC-Fixed maintains a low, smoothly decreasing AND across the full failure range, comparable to SAC-Fixed and heuristic-only (the two architectures explicitly designed or observed to avoid clustering) despite HAC-Fixed being the only architecture that also maintains a high success rate at every failure level – direct evidence that HAC does not purchase its recovery robustness at the cost of the spatial-collapse failure mode identified in Section 4.

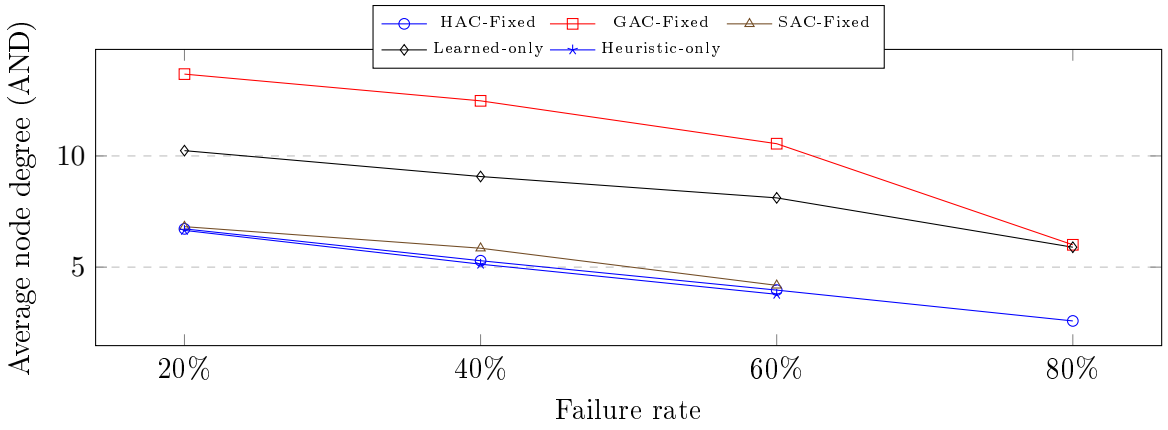


Figure 2: Average node degree (AND) vs. failure rate, random deployment. GAC-Fixed's markedly higher AND reflects the spatial-collapse failure mode of Section 4; HAC-Fixed avoids it while also achieving the highest success rate (Figure 1). SAC-Fixed and heuristic-only omit 80% points where zero trials succeeded.

9 Ablation Studies

9.1 Branch Contribution: Heuristic-Only vs. Learned-Only vs. Fusion

Tables 3 and 4 already contain this ablation: HAC-Fixed (full fusion) against its two constituent branches run in isolation. At 80% failure (random deployment), the fusion reaches 48.8 ± 0.98 successes versus 12.0 ± 3.5 (learned-only) and 0.0 ± 0.0 (heuristic-only); at uniform 60% failure, fusion reaches 50.0 ± 0.0 versus 33.8 ± 6.4 (learned-only) and 0.8 ± 0.7 (heuristic-only). Two observations rule out the two most obvious alternative explanations for HAC's performance. First, the two branches fail in different, non-nested failure ranges (heuristic-only degrades earliest, learned-only more gradually but from a lower ceiling), so fusion cannot be explained as "mostly using whichever branch is better." Second, fusion's success rate at high failure exceeds *both* branches' individual success rates by a wide margin, not merely the better of the two – direct evidence of a genuine synergistic effect rather than redundancy.

9.2 Adaptive vs. Fixed Fusion Weighting

9.2.1 Default Schedule Underperforms Fixed Weighting

Applying R2C's reported schedule ($k = 30$, $s_0 = 0.125$) to HAC-Adaptive *reverses* the direction of R2C's own *w/o AWF* ablation: at 80% failure (random deployment, single seed), HAC-Adaptive reaches only 20/50 successes versus HAC-Fixed's 48.8 ± 0.98 (Table 3).

9.2.2 Schedule Steepness Sweep

Table 6 sweeps (k, s_0) on the random deployment (single training seed per configuration; see the caveat in Section 6.2). The smoothest schedule tested ($k = 5$, $s_0 = 0.7$) nearly closes the gap to HAC-Fixed at 80% failure (48/50 vs. 48.8 ± 0.98/50), and exceeds it slightly on COV and AND (Table 7), while the steepest schedules underperform substantially regardless of their transition center s_0 – consistent with Section 6.2's account that transition *width*, not *timing*, is the operative variable.

Table 6: Schedule steepness sweep, random deployment, single seed per configuration. Success/50 by failure rate.

Schedule (k, s_0)	20%	40%	60%	80%
Default $(30, 0.125)$	50	50	45	20
$(10, 0.125)$	50	50	46	17
$(30, 0.5)$	50	50	43	8
$(10, 0.5)$	50	50	49	34
(5, 0.7)	50	50	**50**	**48**

Table 7: HAC-Fixed vs. HAC with the smoothest schedule (k=5, s_0=0.7) at 80% failure, random deployment.

	REC	COV	AND	Success/50
HAC-Fixed	0.666 ± 0.032	0.604 ± 0.005	2.581 ± 0.023	48.8 ± 0.98
HAC $(5, 0.7)$	0.652	0.619	2.46	48

9.2.3 Cross-Deployment Verification

We verify the $(5, 0.7)$ schedule on uniform deployment (single seed): at 80% failure it reaches 37/50 successes, a substantial improvement over the default schedule (7/50 in preliminary single-seed testing on uniform deployment) though not a full match to HAC-Fixed's 49.0 ± 2.0, suggesting the fix generalizes across deployment scenarios but may benefit from further schedule tuning at the most extreme failure rate.

9.3 Granularity Substitution: HSC as a Counter-Experiment

9.3.1 Experimental Design

To test whether schedule smoothing is a general fix for adaptive fusion or specific to HAC's global learned branch, we construct **HSC**: the same heuristic branch and the same Eq. (3)/ (4) fusion mechanism as HAC, but with the learned branch replaced by SAC's subnetwork-level rigid-translation policy (Section 4.2) – architecturally, the combination closest to R2C's own ISE+IST+AWF design among those we test.

9.3.2 Results: The Fix Does Not Transfer

Table 8 shows that HSC-Fixed itself already collapses at high failure rates (0/50 at 80% failure, both deployments), and – critically – applying the same smoothed schedule that nearly rescued HAC-Adaptive produces almost no improvement for HSC (1/50 random, 0/50 uniform at 80% failure, vs. 0/50 for the default schedule in both cases).

9.3.3 Interpretation

This cleanly separates two distinct failure modes that could otherwise be conflated. HAC's adaptive-schedule failure is a *scheduling* problem: the learned branch has adequate representational capacity (per-node velocities), and a better-shaped handover curve recovers most of the lost robustness. HSC's failure is an *architectural expressiveness* problem: a policy restricted to per-subnetwork rigid translation cannot express the fine-grained corrections needed under severe fragmentation, no

Table 8: HSC (heuristic + subnetwork-level learned branch) does not benefit from schedule smoothing, unlike HAC. Success/50, single seed per configuration.

	Random		Uniform	
Variant	60%	80%	60%	80%
HSC-Fixed	40	0	42	0
HSC-Adaptive (default)	16	0	13	0
HSC-Adaptive $(5, 0.7)$	33	1	40	0

matter how gently or gradually it is granted control. No curriculum can teach a model to compute an output outside its own output space.

9.4 Reproducibility Fix: Seed Control and Its Effect on Baseline Variance

During this study we discovered that our training scripts' `-seed` argument controlled only environment-side randomness (failure selection, initial deployment), leaving PyTorch's network weight initialization unseeded. A single early GAC-Fixed run (uncontrolled initialization) reported 20%-failure COV of 0.830 and AND of 11.87; after fixing weight-initialization seeding and averaging over five properly seeded runs, the same configuration measures COV 0.747 ± 0.009 and AND 13.68 ± 0.28 (Table 3) – a shift larger than one standard deviation, indicating the original single run was an optimistically-biased draw rather than a representative one. We report this as a concrete illustration of why the seeding fix and the subsequent five-seed protocol (Section 7) were necessary before drawing architecture-level conclusions, and log it in full in Appendix A.

10 Discussion

10.1 Why Simple Hybrids Work: Complementary Failure Modes Revisited

The through-line connecting Sections 4, 8, and 9.1 is that GAC's and SAC's failure modes are complementary at the level of *individual trajectories*, not merely on average: the heuristic branch provides a locally certifiable descent direction on connectivity (Proposition 1) whenever it is in the attractive zone, and the learned branch's global receptive field lets it act on information no purely local rule can see (e.g., the relative position of a distant, disconnected component). Neither alone is sufficient, but their weaknesses do not overlap in the same states at the same time, which is precisely the condition under which a fusion – even a naively simple, fixed-weight one – outperforms either constituent.

10.2 When Schedule Smoothing Helps and When It Doesn't

Section 9 shows this benefit is conditional: schedule smoothing rescues HAC (whose learned branch has per-node representational capacity) but not HSC (whose learned branch is restricted to per-subnetwork translation). We read this as a general design principle for adaptive fusion of heuristic and learned components: *before* tuning the fusion schedule, verify that the learned branch's output space is rich enough to express the corrections the task demands at the granularity the fusion schedule will eventually hand it full control over. Tuning a schedule cannot substitute for representational capacity the architecture does not have.

10.3 Limitations

10.3.1 Simplified Learned Branch

HAC's global graph convolution omits R2C's convex-hull boundary-node extraction, virtual cross-subgraph edge construction, and larger (8-layer, 512-hidden) spectral convolution. This is by design (Section 5), but it means the remaining 9–11 percentage-point coverage gap at 80% failure (Section 8.2) cannot currently be closed without reintroducing some of this machinery, and we have not yet identified which specific piece (if any) is responsible for the gap.

10.3.2 Non-Controlled Comparison with R2C

R2C's code is not publicly released. Our comparison in Section 8.2 uses numbers as published, under an independently re-implemented simulation environment; low-failure-rate comparisons in particular should be read as indicative rather than precise, per the discussion in Section 8.2.

10.3.3 Truncated Training Horizon

All models are trained on 40-step differentiable rollouts but evaluated on the full 400-step recovery horizon, a tractability trade-off for backpropagation-through-time training. We have not measured whether longer training rollouts would change any of the reported comparisons.

10.3.4 Single-Seed Schedule and HSC Sweeps

The schedule steepness sweep (Table 6) and the HSC counter-experiment (Table 8) are each run with a single training seed per configuration, unlike the five-seed main results (Tables 3–4). Given the five-seed variance measured for comparable architectures (success-count standard deviations up to ±5–6 out of 50), fine-grained orderings within these sweeps – but not their headline conclusions, which are corroborated by cross-deployment verification and by the theoretical account of Section 6 – should be treated as preliminary.

10.4 Future Work

Three directions follow directly from the limitations above: (i) test the falsifiable autonomy-exposure prediction of Section 6.3 with a multi-seed schedule sweep; (ii) identify, via targeted ablation of R2C's IST machinery, which specific component (boundary-node extraction vs. multipartite edges vs. network capacity) is responsible for R2C's remaining coverage advantage; and (iii) explore lightweight ways to grant a subnetwork-level learned branch node-level expressiveness – e.g., a small per-node residual on top of the subnetwork translation – to test whether this closes HSC's gap without reverting to a fully global learned branch.

11 Conclusion

We set out to answer two questions R2C's own ablations leave open: does its subnetwork-level learned component need to be subnetwork-level, and does its adaptive fusion schedule need its specific reported parameters? Through a sequence of controlled experiments – motivating single-branch failure modes, a deliberately minimal hybrid architecture (HAC), a schedule-steepness sweep, and a granularity-substitution counter-experiment (HSC) – we find that the answer to both questions is more nuanced than either "yes" or "no." A single global learned branch, fused with a parameter-free heuristic branch under fixed weighting, already recovers the large majority of R2C's benefit with

markedly higher robustness than any single-branch alternative we test. R2C's adaptive schedule, however, does not transfer as-is to this simpler architecture; the failure is one of transition steepness rather than adaptive weighting per se, is fixable by smoothing the schedule, and – critically – that fix is itself architecture-dependent, transferring to a node-level learned branch but not to a subnetwork-level rigid-translation one. We report this account, including a falsified hypothesis, a documented numerical bug, and several single-seed results explicitly flagged as preliminary, in the interest of giving future work an honest and reproducible starting point rather than a polished but incomplete one.

References

Ameer A. Abbasi and Mohamed Younis. Movement-assisted connectivity restoration in wireless sensor and actor networks. *IEEE Transactions on Parallel and Distributed Systems*, 20(9):1366–1379, 2008.

Kemal Akkaya, Fatih Senel, Aravind Thimmapuram, and Suleyman Uludag. Distributed recovery from network partitioning in movable sensor/actor networks via controlled mobility. *IEEE Transactions on Computers*, 59(2):258–271, 2009.

C. Bradford Barber, David P. Dobkin, and Hannu Huhdanpaa. The quickhull algorithm for convex hulls. *ACM Transactions on Mathematical Software*, 22(4):469–483, 1996.

Ming Chen, Hao Wang, Chih-Yung Chang, and Xin Wei. SIDR: A swarm intelligence-based damage-resilient mechanism for UAV swarm networks. *IEEE Access*, 8:77089–77105, 2020.

Andrea Coletta, Francesco Giorgi, Gaia Maselli, Marco Prata, Daniel Silvestri, Jonathan Ashdown, and Francesco Restuccia. A 2-UAV: Application-aware content and network optimization of edge-assisted UAV systems. In *IEEE INFOCOM 2023 - IEEE Conference on Computer Communications*, pages 1–10, 2023.

Stefan Elfwing, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018.

Zhiwei Guo, Yifan Sun, Yue Wang, Chen Fu, and Jing Zhong. An intelligent framework for resilience recovery of FANETs with spatio-temporal aggregation and multi-head attention mechanism. *Computers, Materials & Continua*, 79(2):2375–2398, 2024.

Yogesh K. Joshi. Autonomous recovery from multi-node failure in wireless sensor networks. In *2012 IEEE Global Communications Conference (GLOBECOM)*, pages 652–657, 2012.

Nikolaos Karalias and Andreas Loukas. Erdos goes neural: An unsupervised learning framework for combinatorial optimization on graphs. *Advances in Neural Information Processing Systems*, 33: 6659–6672, 2020.

Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *arXiv preprint arXiv:1609.02907*, 2016.

Harrshavasan Kurunathan, Hailong Huang, Kai Li, Wei Ni, and Ekram Hossain. Machine learning-aided operations and communications of unmanned aerial vehicles: A contemporary survey. *IEEE Communications Surveys & Tutorials*, 26(1):496–533, 2023.

Jian Li, Ping Yi, Tong Duan, Ze Zhang, Jing Li, Yue Wang, and Jian Yu. Fast connectivity restoration of UAV communication networks based on distributed hybrid MADDPG and APF algorithm. *Ad Hoc Networks*, 171:103785, 2025.

Jun Li, Cheng Zhou, Jia Liu, Min Sheng, Nan Zhao, and Yi Su. Reinforcement learning-based resource allocation for coverage continuity in high dynamic UAV communication networks. *IEEE Transactions on Wireless Communications*, 23(2):848–860, 2023.

Hao Lin and Liang Ding. Multi-hop differential topology based algorithms for resilient network of UAV swarm. *arXiv preprint arXiv:2411.11342v2*, 2025.

Hao Lin, Liang Ding, Song Chen, Fan Yang, and Ling Qian. Multi-hop diffused graph convolution for resilient UAV swarm networks. In *2024 IEEE International Symposium on Broadband Multimedia Systems and Broadcasting (BMSB)*, pages 1–6, 2024.

Fang Liu and Qing Wang. Asymmetric learning for spectral graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 18798–18806, 2025.

Zhangfeng Ma, Bo Ai, Ruisi He, Guangjie Wang, Yong Niu, and Zhangdui Zhong. A wideband non-stationary air-to-air channel model for UAV communications. *IEEE Transactions on Vehicular Technology*, 69(2):1214–1226, 2019.

Zhenqiang Mi and Jenny Y. Yang. Obstacle-avoidance connectivity restoration for mobile sensor systems with local information. In *2015 IEEE International Conference on Communications (ICC)*, pages 6395–6399, 2015.

Zhenqiang Mi, Yang Yang, and Guangming Liu. HERO: A hybrid connectivity restoration framework for mobile multi-agent networks. In *2011 IEEE International Conference on Robotics and Automation*, pages 1702–1707, 2011.

Zhiyu Mou, Feifei Gao, Jian Liu, and Qing Wu. Resilient UAV swarm communications with graph convolutional neural network. *IEEE Journal on Selected Areas in Communications*, 40(1):393–411, 2022.

Dor Mutzari, Yonatan Aumann, and Sarit Kraus. Heterogeneous multi-robot graph coverage with proximity and movement constraints. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 14646–14654, 2025.

Hao Ni, Qiuming Zhu, Bo Hua, Kai Mao, Yuxin Pan, Farman Ali, Weizhi Zhong, and Xiaomin Chen. Path loss and shadowing for UAV-to-ground UWB channels incorporating the effects of built-up areas and airframe. *IEEE Transactions on Intelligent Transportation Systems*, 25(11): 17066–17077, 2024.

Yabin Peng, Chang Liu, Shaoxun Liu, Yuan Liu, and Yue Wu. SmartTRO: Optimizing topology robustness for internet of things via deep reinforcement learning with graph convolutional networks. *Computer Networks*, 218:109385, 2022.

Yabin Peng, Jun Wu, Tong Duan, Yuan Liu, Ze Zhang, and Jie Zhang. Prioritized recovery strategy for robust UAV swarm communication via graph reinforcement learning. *IEEE Internet of Things Journal*, 12(13):23891–23904, 2025.

Yabin Peng, Chenyu Zhou, Hainan Cui, Tong Duan, Haoyang Chen, Fan Zhang, and Shaoxun Liu. Resilient UAV swarm with fast connectivity recovery and extensive coverage. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 917–925, 2026.

Jan Rychlewski. On hooke's law. *Journal of Applied Mathematics and Mechanics*, 48(3):303–314, 1984.

Ran Shi, Xin Yu, Yue Wang, Ye Tian, Zhi Liu, Wei Wu, Xiao-Ping Zhang, and Manuela M. Veloso. Symmetry-informed MARL: A decentralized and cooperative UAV swarm control approach for communication coverage. *IEEE Transactions on Mobile Computing*, 24(9):8039–8056, 2025.

Siddharth Shriwastav. Round-table negotiation for fast restoration of connectivity in partitioned wireless sensor networks. *Ad Hoc Networks*, 77:11–27, 2018.

Mehmet Tosun, Umut Can Cabuk, Emre Haytaoglu, Orhan Dagdeviren, and Yusuf Ozturk. DP-kCR: Distributed proactive k-connectivity recovery algorithm for UAV-based MANETs. *IEEE Transactions on Reliability*, 73(4):1918–1932, 2024.

Jian Wang, Shuo Li, Di Chen, Qiang Fu, Qi Qi, Hao Sun, Bo He, and Jianxin Liao. Flight trajectory control with network-oriented hierarchical reinforcement learning for UAV-assisted time-sensitive IoT. *IEEE Transactions on Intelligent Transportation Systems*, 26(5):6332–6345, 2025a.

Xu Wang, Zhen Zhao, Liang Yi, Zhaolong Ning, Lei Guo, F. Richard Yu, and Song Guo. A survey on security of UAV swarm networks: Attacks and countermeasures. *ACM Computing Surveys*, 57(3):1–37, 2024a.

Yao Wang, Wei Chen, Tom H. Luan, Zhou Su, Qichao Xu, Ru Li, and Ning Chen. Task offloading for post-disaster rescue in unmanned aerial vehicle networks. *IEEE/ACM Transactions on Networking*, 30(4):1525–1539, 2022.

Yingze Wang, Zhen Feng, Hui Zhang, Yang Gao, Jing Lei, Le Sun, and Ming Song. Angle robustness unmanned aerial vehicle navigation in GNSS-denied scenarios. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 10386–10394, 2024b.

Yue Wang, Xu Wang, Chao Wei, Qi Ren, and Yong Tang. UAV swarm network topology self-healing via graph-based deep reinforcement learning. In *2025 IEEE Wireless Communications and Networking Conference (WCNC)*, pages 1–6, 2025b.

Zhi Wang, Xin Zhao, Jie Zhang, Nan Yang, Peng Wang, Jian Tang, Jun Zhang, and Liang Shi. APF-CPP: An artificial potential field based multi-robot online coverage path planning approach. *IEEE Robotics and Automation Letters*, 9(11):9199–9206, 2024c.

Chejian Xu, Wenhao Ding, Weijie Lyu, Zuxin Liu, Shuai Wang, Yihan He, Hanjiang Hu, Ding Zhao, and Bo Li. SafeBench: A benchmarking platform for safety evaluation of autonomous vehicles. *Advances in Neural Information Processing Systems*, 35:25667–25682, 2022.

Zhiqiang Ye, Ke Wang, Yu Chen, Xiang Jiang, and Guanghua Song. Multi-UAV navigation for partially observable communication coverage by graph reinforcement learning. *IEEE Transactions on Mobile Computing*, 22(7):4056–4069, 2022.

Dingyi Zeng, Wanlu Liu, Wenjie Chen, Liang Zhou, Miao Zhang, and Hua Qu. Substructure aware graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 37, pages 11129–11137, 2023.

Lei Zhang, Yi Du, Jing Xu, and Xin Wang. UAV-enabled IoT: Cascading failure model and topology-control-based recovery scheme. *IEEE Internet of Things Journal*, 11(12):22562–22577, 2024.

A Deviations Log

We log every point at which our implementation deviates from Peng et al. (2026)'s stated methodology, or from our own earlier implementation, in the interest of full transparency about what was actually measured.

- **Failure-node selection.** R2C states only "unpredictable random failure patterns" without specifying the selection algorithm; we use uniform random node removal.
- **Connectivity loss surrogate.** R2C's own connectivity loss (its Eq. 19) counts connected components, which is non-differentiable. We use the soft-minimum surrogate of Eq. (5).
- **Count-bias bug and fix.** An earlier version of Eq. (5) omitted the $\log(n_i n_j)$ correction term. Because plain log-sum-exp is not count-invariant, this systematically under-penalized distance between large disconnected components independent of their true spatial separation, occasionally producing spuriously negative training losses. We verified the fix numerically: on a controlled synthetic case with matched true minimum distance ($\approx$ 100–108 m) but mismatched pair counts (3×3 vs. 30×30), the uncorrected estimator returned 2.85 and -230.58 respectively (wildly count-dependent), while the corrected estimator returned 112.71 and 109.54 (both close to the true minimum, as intended). All results in this paper use the corrected estimator; results computed before the fix are not reported.
- **Boundary buffer width d_e.** Not numerically specified by R2C; we use $d_e = 0.15R_c$, the same order of magnitude as the neutral zone width $d_u - d_l = 0.1R_c$.
- **Truncated training rollout.** Training uses 40-step rollouts versus the full 400-step evaluation horizon, for backpropagation-through-time tractability (Section 10).
- **Coverage computed via Monte Carlo grid sampling**, at 5 m resolution, rather than R2C's (unspecified) exact implementation.
- **Weight-initialization seeding bug and fix.** Documented in Section 9.4; the `-seed` argument originally controlled only environment-side randomness. Fixed by explicitly calling `torch.manual_seed` and `torch.cuda.manual_seed_all` before model construction in every training script, prior to the five-seed studies reported in this paper.
- **R2C code unavailable.** No official implementation is released; Section 8.2's comparison uses R2C's published numbers directly and is explicitly not a controlled reproduction.

B Full Hyperparameter Tables

C Additional Result Tables

Tables 3 and 4 in the main text already report full REC/COV/Success statistics across all four failure rates for all five architectures and both deployment scenarios; we do not duplicate them here. AND values for the uniform deployment, omitted from the main text for space, are: HAC-Fixed $5.72 \to 4.32 \to 3.30 \to 2.37$; GAC-Fixed $5.88 \to 6.86 \to 8.75 \to 5.81$; SAC-Fixed $5.74 \to 4.46 \to$ n/a $\to$ n/a; Learned-only $5.76 \to 6.37 \to 7.34 \to 5.29$; Heuristic-only $5.72 \to 4.36 \to 2.87 \to$ n/a (in order of $20\%, 40\%, 60\%, 80\%$ failure).

Table 9: Hyperparameters used across all trained architectures unless otherwise noted.

Parameter	Value
Graph convolution layers L	4
Hidden dimension	128
k-NN degree	8
Optimizer	Adam
Learning rate	1×10^{-3}
Gradient norm clip	1.0
Training epochs	300
Training rollout length	40 steps
Evaluation horizon T	400 steps
Movement cost coefficient λ	0.01
α, β, δ (heuristic branch)	0.01
$[d_l, d_u]$	$[0.8R_c, 0.9R_c]$
$R_c = R_{\text{cov}}$	120 m
$\hat{v}_{\max}$	1 m/s
Mission area	$1000 \times 1000\,\text{m}^2$
Random deployment UAV count	200
Uniform deployment UAV count	196 (14×14 grid)

D Code and Reproducibility

D.1 Repository Structure

The released code covers the HAC main line reported in this paper (Sections 5–9): the simulation environment, the heuristic and learned branches, training and evaluation scripts, the branch-contribution ablation, and the multi-seed study and aggregation script. It does not include the GAC/SAC motivating-experiment code (Section 4) or the HSC counter-experiment (Section 9.3) as separate release artifacts, though the results from both are reported in full in this paper from the original experimental logs.

D.2 Setup and Reproduction

A `conda` environment specification, a resumable multi-seed sweep script, and a results-aggregation script that reproduces Tables 3 and 4 directly from raw per-seed evaluation logs are included with the code release; see the accompanying `README` for exact commands.